

阶段一：Question_Evolution Agent 改造实施计划

阶段一：Question_Evolution Agent 改造实施计划（按当前项目校准版）

状态：已实施

更新日期：2026-08-05

适用范围：当前 QA_Evolution_Agent 仓库

目标读者：后续实现人员、人工 reviewer、实验负责人

1. 执行结论

当前项目已经不是早期的单链路题目进化脚本，而是一个可恢复、可审计的实验流水线。它已经具备多算子横向搜索、两算子纵向叠加、运行期 policy、schema、manifest、实验统计和局部 Memory 写入能力。

因此，Agent 改造的正确落点不是重写流水线，也不是新建一套通用多 Agent 框架，而是在现有流水线外新增一层“受控实验 Agent 外壳”。它负责把用户目标转成计划，检查环境，按权限调用已登记工具，观察产物，做停止或复盘判断，并输出报告。

第一版必须包含四个能力：

1. **规划**：从 AgentTask 生成可审计的 AgentPlan。
2. **执行**：通过工具封装调用 check_runtime_environment.py、run_loop.sh、resume_run_loop.sh。
3. **工具调用记录**：每次工具调用都落 agent_events.jsonl，失败可定位。
4. **记忆**：新增本次 Agent 运行记忆 M0，读取现有实验 Memory M1 摘要；不发布 active 全局 Memory。

第一版不做：

- 不重写 run_loop.sh 主流程。
- 不拆 multi_operator_pipeline.py 或 vertical_search_pipeline.py 的内部阶段。
- 不替换 operator_router.py、question_evolution.py、scoring.py、pipeline_io.py。
- 不让 Agent 自动修改正式 Prompt、Router、Rubric、算子内容或 schema。

- 不让 Memory 直接决定算子、跳过 Router、跳过 manifest 校验或绕过 `execution_scope`。
- 不把自动评分结果写成“已确认有效边界”。
- 不接入跨项目长期 Memory、向量库、LangChain、LangGraph 或递归多 Agent。

2. 当前项目真实基线

2.1 已有主入口

`run_loop.sh` 当前支持三种 `SEARCH_MODE`：

代码块

```
1  single_branch
2  multi_operator_branch
3  multi_operator_vertical_stack
```

其中 `multi_operator_branch` 是默认模式；`multi_operator_vertical_stack` 已由 `vertical_search_pipeline.py` 实现；`single_branch` 是保留的跨轮单主链模式。

2.2 已有搜索与路由能力

2.3 已有数据契约与断点恢复

`pipeline_io.py` 已经提供：

- JSON/JSONL 读取兼容；

- `write_stage_jsonl()` 和 `manifest`;
- `.partial` 恢复;
- `published_artifact_valid()` ;
- sidecar 校验;
- cross-process 文件锁。

Agent 工具层必须复用这些机制判断“产物可复用 / 产物损坏 / 需要停止”，不能自己用简单的“文件存在”判断覆盖产物。

现有 schemas 已覆盖核心流水线字段：

2.4 已有 Memory 现状

当前项目已经有实验级 append-only Memory：

代码块

```
1 <experiment_dir>/memory/operator_memory_bank.jsonl
2 <experiment_dir>/memory/failure_memory_bank.jsonl
3 <experiment_dir>/memory/invalid_generation_cases.jsonl
4 <experiment_dir>/memory/.memory_idempotency.jsonl
```

这些文件由 `update_sample_state.py` 写入，并通过 idempotency ledger 避免重复追加。它们是局部实验经验，不是全局 active 策略库。

第一版 Agent 新增的 Memory 只分两类：

作用是审计和恢复。比如之后发现实验没跑完，可以看 `agent_events.jsonl` 知道停在哪个工具；看 `agent_plan.json` 知道当时 Agent 准备怎么做。

跨实验全局 Memory、active 策略库、proposal 到 active 的发布流程放到第二版以后。第一版最多写 `proposed` 或 `needs_human_review` 类型的复盘建议。

3. 当前实施约束

第一版实现按当前项目代码和运行产物落地，直接遵守以下约束：

- `execution_scope` 只能使用 `full_iteration`、`reference_rebuild_only`、`debug_generation_only`；只读复盘使用独立的 `review_mode=report_only` 表达，不能混入 `execution_scope`。
- Agent 运行文件先写入 `agent_runs/<date>/<agent_run_id>/`；新实验由 `run_loop.sh` 创建真实 `experiments/<date>/exp*` 目录后，Agent 再记录该目录并可复制报告。
- Observer 优先读取 `experiment_statistics.json/txt`、`summary.txt` 和最终产物；统计不足时再回退读取 `round_*/state_updated.jsonl`、`branch_results.jsonl`、`exploration_candidates.jsonl`、`effect_analysis.jsonl` 和纵向搜索摘要。
- 第一版只写 M0 Agent 运行记忆，并只读 M1 实验 Memory 摘要；不发布 active 全局 Memory，不让 Memory 改变 Router candidate set、执行顺序或已冻结的 `operator_plan`。
- Global Judge 第一版只做只读复盘报告和 proposal 输出；不得修改正式 Prompt、Router、Rubric、Memory、score、state 或算子文件。
- 多 Agent 协作、跨实验全局 Memory active 发布、Shadow/Replay 自动验证和策略发布治理均不作为第一版依赖；需要时单独制定当前项目版实施计划。

4. 目标架构

4.1 运行链路

代码块

```
1  AgentTask
2  -> 任务校验
3  -> Context Pack 构造
4  -> AgentPlan 生成
5  -> Policy Guard 校验计划
6  -> Tool Registry 调用现有入口
7  -> Observer 读取产物摘要
8  -> Decision Engine 判断停止、续跑、复盘或阻塞
9  -> Reporter 输出 agent_report.md
```

Agent 只决定“调用哪个登记工具、使用哪些参数、何时停止、如何复盘”。题目画像、路由、生成、校验、回答、评分、效果分析、分支调度和实验 Memory 继续由现有流水线负责。

4.2 新增文件建议

第一版新增文件控制在最小可维护范围内：

代码块

```
1  question_evolution_agent.py
2  agent_runtime/
3    __init__.py
4    task.py
5    planner.py
6    context.py
7    policy.py
8    tools.py
9    observer.py
10   decisions.py
11   events.py
12   state.py
13   reporter.py
14   schemas/
15     agent_task.schema.json
16     agent_plan.schema.json
17     agent_run_state.schema.json
18     agent_tool_event.schema.json
19     agent_observation.schema.json
20     agent_decision.schema.json
```

暂不新增 `agent_runtime/memory.py`、`agent_runtime/judge.py` 或多 Agent runtime。Memory 摘要先放在 `observer.py`，只读复盘先放在 `reporter.py` 或后续单独拆分，避免第一版过度拆文件。

4.3 文件职责

5. AgentTask 契约

每次运行从 AgentTask 开始。它描述目标和边界，不描述每一步怎么实现。

示例：

代码块

```
1  {"goal": "在本批样本中找到 5 条高可信降分边界候选","input_file":
   "data/police_qa_testset_4scenes_sub.jsonl","search_mode":
   "auto","boundary_target": 5,"max_search_steps": 25,"execution_scope":
   "full_iteration","review_mode": "none","planning_mode":
   "deterministic","allowed_tools": ["check_environment", "run_full_loop",
   "observe_experiment", "write_agent_report"],"allow_prompt_mutation":
   false,"allow_memory_active_publish": false,"allow_global_memory_read":
   false,"exp_root": "experiments","resume_exp_dir": "", "resume_start_round":
   null}
```

字段规则：

5.1 search_mode=auto 规则

确定性规划先按以下规则选择：

如果目标文本无法判断，默认 multi_operator_branch ，并在 AgentPlan.assumptions 写明。

6. AgentPlan 契约

AgentPlan 必须写入：

代码块

```
1 agent_plan.json
```

必填内容：

```
1 plan_id
2 goal_summary
3 selected_search_mode
4 selected_execution_scope
5 selected_review_mode
6 budget
7 env_overrides
8 steps
9 assumptions
10 blocked_reasons
```

每个 step 必须包含：

代码块

```
1 step_id
2 tool
3 purpose
4 inputs
5 expected_outputs
6 stop_if_failed
7 run_when
```

第一版典型新实验计划：

代码块

```
1 1. check_environment
2 2. run_full_loop
3 3. observe_experiment
4 4. decide_next_action
5 5. write_agent_report
```

典型续跑计划：

代码块

```
1 1. locate_resume_root
2 2. resume_full_loop
3 3. observe_experiment
4 4. decide_next_action
5 5. write_agent_report
```

典型只读复盘计划：

代码块

```
1 1. observe_experiment
2 2. write_agent_report
```

7. Tool Registry

Tool 是实际执行动作，Skill 只是 Agent 的能力分类。O10-O33 是业务算子，不是 Agent Skill。

第一版工具：

7.1 工具调用规则

- 调用前写 `tool_started`。
- 成功后写 `tool_completed`，记录 return code、耗时、stdout 摘要、关键产物路径。
- 失败后写 `tool_failed`，记录 return code、stderr 摘要、是否可恢复。
- 不能把完整 stdout、完整 JSONL 或完整模型响应写进 `agent_events.jsonl`。
- 不能把 API key、Base URL、Authorization、环境变量全量写进事件。
- 工具执行目录固定为项目根目录。
- 工具只允许设置白名单环境变量，例如 `INPUT_FILE`、`EXP_ROOT`、`SEARCH_MODE`、`BOUNDARY_TARGET`、`MAX_SEARCH_STEPS`、`EXECUTION_SCOPE`、纵向搜索预算和并发控制项。

7.2 新实验目录处理

当前 `run_loop.sh` 在新实验时自行创建 `EXP_ROOT/<date>/exp*`。第一版 Agent 不强改这一行为。

实现要求：

1. Agent 先创建自己的运行目录：

代码块

```
1 agent_runs/<date>/<agent_run_id>/
```

1. 在该目录写 M0 文件。
2. 调用 `run_loop.sh` 时设置 `EXP_ROOT`。
3. 从 stdout 的“本次实验目录:”或 `EXP_ROOT` 下最新 manifest 完整目录定位真实实验目录。
4. 将真实实验目录写入 `agent_run_state.json`。
5. 报告最终写在 Agent run 目录；如果真实实验目录已定位成功，可额外复制一份到实验目录，但不能覆盖已有正式流水线产物。

后续如果要想让 `run_loop.sh` 支持新实验 `EXP_DIR` 精确指定，必须作为独立小改提交并配套测试，不作为 Agent 第一版前置条件。

8. Context Pack

`Context Pack` 是给 planner/decision 层的短上下文。第一版即使不接控制层模型，也要按这个结构保存，便于后续接入模型。

包含：

代码块

- 1 用户目标
- 2 任务配置
- 3 当前项目硬约束摘要
- 4 计划使用的 `search_mode` / `execution_scope`
- 5 预算与停止条件
- 6 实验目录与已发布产物列表
- 7 Observation 摘要
- 8 Memory 摘要
- 9 可调用工具列表
- 10 上一次决策

不包含：

代码块

- 1 完整 JSONL
- 2 完整日志
- 3 完整模型响应
- 4 API key 或 Base URL
- 5 旧参考答案全文
- 6 旧 Rubric 全文
- 7 隐藏答案材料

超过上下文预算时保留优先级：目标、停止条件、失败原因、边界候选统计、pending 统计、manifest 状态、复盘建议。

9. Observer 规则

Observer 的输入是实验目录，输出 `agent_observation.json`。

优先读取：

代码块

- 1 summary.txt
- 2 experiment_statistics.json
- 3 experiment_statistics.txt
- 4 final/final_scored.jsonl
- 5 round_*/state_updated.jsonl
- 6 round_*/branch_results.jsonl
- 7 round_*/exploration_candidates.jsonl
- 8 round_*/effect_analysis.jsonl
- 9 round_*/quantity_metrics_summary.json
- 10 memory/*.jsonl

必须统计：

产物缺失时，Observer 返回缺失清单，不抛出难读异常。只有目录不存在、JSON 无法解析或 manifest 明确损坏时才标记为 `blocked`。

10. Decision Engine

允许的决策：

确定性停止条件：

- `boundary_candidate_count >= boundary_target`。
- `pending_count=0`。
- `max_search_steps` 已由流水线消耗或搜索状态显示预算耗尽。
- `run_loop.sh` 正常结束并已生成 `final/final_scored.jsonl`。
- `review_mode=report_only` 且报告已生成。
- 工具失败且没有可恢复路径。
- manifest 损坏或 `.partial` 无法恢复。

- 计划或决策要求越权。

第一版不做自动多轮“Agent 自己决定再发起下一次全新实验”。如果需要二次实验，报告中写建议，由人工确认后再提交新的 AgentTask。

11. Policy Guard

agent_runtime/policy.py 默认拒绝：

被拒绝时写入 agent_events.jsonl 和 agent_report.md。

12. 报告要求

agent_report.md 必须包含：

- 本次目标。
- 输入文件或续跑目录。
- Agent run 目录。
- 真实 experiment 目录。
- 搜索模式、执行范围、预算。
- 调用过的工具和结果。
- 主要产物路径。
- 边界候选数量。
- score_increased、not_applicable、validation_failed、branch_error 统计。
- 是否达到目标。

- 是否需要人工审核。
- 下一步建议。

报告只能写：

代码块

- 1 边界候选
- 2 自动评分证据
- 3 需要人工确认
- 4 复盘建议

不能写：

代码块

- 1 已确认有效边界
- 2 已证明模型能力边界
- 3 已自动修复算子
- 4 已发布 Memory

13. 只读复盘与 Global Judge 边界

第一版的复盘不是自进化闭环，只是只读报告。

输出可以包括：

代码块

- 1 global_review_report.md
- 2 optimization_proposals.jsonl

复盘回答：

1. 本次实验主要失败层级是什么。
2. 哪些算子经常 `not_applicable`。
3. 哪些分支经常 `score_increased`。
4. 哪些候选需要人工审核。
5. 哪些 proposal 值得进入后续 Shadow/Replay。

proposal 状态只能是：

代码块

```
1 proposed
2 needs_human_review
3 rejected_insufficient_evidence
```

第一版不允许复盘自动修改正式 Prompt、Router、Rubric、Memory、score、state 或算子文件。

14. 实施阶段

阶段 0：基线确认与文档校准

目标：确保 Agent 改造基于当前真实代码和项目约束。

工作：

- 阅读 `AGENTS.md`、`全局规则约束.md`、`README.md`。
- 确认 `run_loop.sh`、`resume_run_loop.sh`、`check_runtime_environment.py` 的当前参数。
- 确认 `execution_scope.py` 的真实枚举。
- 确认 `schemas/` 和 `memory/` 当前语义。

验收：

- `execution_scope` 字段只使用当前代码支持的三个枚举。
- Memory、Global Judge 和多 Agent 扩展不作为第一版前置依赖。
- 后续实现不与无关文档删除或移动混在同一个 diff 中。

阶段 1：AgentTask、状态、事件和 policy 骨架

新增：

代码块

```
1 question_evolution_agent.py
2 agent_runtime/task.py
3 agent_runtime/state.py
4 agent_runtime/events.py
5 agent_runtime/policy.py
6 schemas/agent_task.schema.json
7 schemas/agent_run_state.schema.json
8 schemas/agent_tool_event.schema.json
```

验收：

- 能解析 `AgentTask`。

- 缺少 `goal`、新实验缺少 `input_file`、续跑缺少 `resume_exp_dir` 时返回清晰错误。
- 非法 `search_mode`、`execution_scope`、`review_mode` 被拒绝。
- 能创建 `agent_runs/<date>/<agent_run_id>/`。
- 能写 `agent_task.json`、`agent_run_state.json`、`agent_events.jsonl`。
- 不访问真实 API 也能通过测试。

阶段 2：确定性 planner 和 dry-run

新增：

代码块

```
1 agent_runtime/planner.py
2 agent_runtime/context.py
3 schemas/agent_plan.schema.json
```

验收：

- `python question_evolution_agent.py dry-run --task <task.json>` 能输出 `AgentPlan`。
- `search_mode=auto` 能按目标选择运行模式。
- 复盘目标不会生成新实验计划。
- 每个步骤都绑定已登记 Tool。
- `AgentPlan` 写明 env overrides 和 assumptions。
- 计划要求越权时被 `policy.py` 拦截。

阶段 3：工具封装

新增：

代码块

```
1 agent_runtime/tools.py
```

封装：

代码块

```
1 check_environment
2 run_full_loop
```



```
3 resume_full_loop
```

验收：

- mock subprocess 时能记录 `tool_started`、`tool_completed`、`tool_failed`。
- `check_environment` 使用 `--json` 并解析 ready 状态。
- `run_full_loop` 能设置 `EXP_ROOT`、`INPUT_FILE`、`SEARCH_MODE`、预算和 `EXECUTION_SCOPE`。
- `resume_full_loop` 能调用 `resume_run_loop.sh <exp_dir> <start_round>`。
- 能从 stdout 或目录扫描定位真实实验目录。
- stderr/stdout 写摘要，不写完整日志和凭据。

阶段 4：产物观察和报告

新增：

代码块

```
1 agent_runtime/observer.py
2 agent_runtime/reporter.py
3 schemas/agent_observation.schema.json
```

验收：

- 能读取 `experiment_statistics.json`。
- 能统计 branch status、pending、boundary、validation failure 和 Memory 摘要。
- 产物缺失时输出缺失清单。
- 能输出 `agent_report.md`。
- 报告不写“确认有效边界”。
- 小型 fixture 覆盖横向、纵向、single_branch、缺文件和损坏 JSON 场景。

阶段 5：决策引擎

新增：

代码块

```
1 agent_runtime/decisions.py
2 schemas/agent_decision.schema.json
```

验收：

- 达到 `boundary_target` 时 `stop_and_report`。
- `pending_count=0` 时 `stop_and_report`。
- manifest 损坏时 `blocked`。
- 工具失败且不可恢复时 `blocked`。
- 只读复盘完成时 `stop_and_report`。
- 决策写入 `agent_decisions.jsonl`。

阶段 6：CLI 串联 run/resume/review

修改：

代码块

```
1 question_evolution_agent.py
2 agent_runtime/state.py
3 agent_runtime/reporter.py
```

验收：

- `run --task <task.json>` 能完成：校验、规划、执行、观察、报告。
- `resume --task <task.json>` 能续跑已有实验。
- `review --exp-dir <dir>` 只读生成报告，不启动模型调用。
- 所有命令能在 subprocess mock 下测试。
- 原有 `bash run_loop.sh` 和 `bash resume_run_loop.sh` 仍能独立运行。

阶段 7：控制层模型接入（后置）

只有前六阶段稳定后再接入控制层模型。

新增或修改：

代码块

```
1 agent_runtime/planner.py
2 agent_runtime/context.py
3 agent_runtime/decisions.py
```

新增配置：

代码块

```
1  AGENT_MODEL=""
2  AGENT_BASE_URL=""
3  AGENT_API_KEY=""
4  AGENT_TEMPERATURE=0
5  AGENT_TIMEOUT=120
6  AGENT_MAX_CONTEXT_CHARS=60000
```

验收：

- 默认仍可在无 `AGENT_MODEL` 时使用确定性规划。
- 模型输入只使用 `Context Pack`。
- 模型输出必须通过 schema。
- schema 不通过时走确定性兜底。
- 单元测试全部 mock，不访问真实 API。

阶段 8：只读复盘 proposal

新增或扩展：

代码块

```
1  agent_runtime/reporter.py
```

验收：

- 输出 `global_review_report.md`。
- 输出 `optimization_proposals.jsonl`。
- 每条 proposal 有 evidence refs。
- proposal 状态只允许 `proposed`、`needs_human_review`、`rejected_insufficient_evidence`。
- 正式 Prompt、Memory、score、state 不变化。

15. 测试要求

新增测试建议：

代码块

```
1  tests/test_agent_task.py
2  tests/test_agent_policy.py
3  tests/test_agent_planner.py
4  tests/test_agent_tools.py
```

```
5 tests/test_agent_observer.py
6 tests/test_agent_decisions.py
7 tests/test_agent_reporter.py
8 tests/test_agent_cli.py
```

测试原则：

- 不访问真实 API。
- 模型调用全部 mock。
- subprocess 调用全部 mock。
- 产物读取使用小型 fixture。
- policy、schema、observer、decision 必须有负例。
- 不改变现有 `tests/test_stage06_run_loop_integration.py` 对 `run_loop.sh` 的约束。

必须覆盖：

1. `AgentTask` 缺字段。
2. 非法 `search_mode`。
3. 非法 `execution_scope`。
4. `review_mode=report_only` 不触发真实运行。
5. `run_full_loop` 成功定位实验目录。
6. `resume_full_loop` 参数缺失时拒绝。
7. manifest 损坏时拒绝继续。
8. 控制层模型输出非法 JSON。
9. 越权修改 Prompt。
10. 越权发布 active Memory。
11. pending 为 0 的停止判断。
12. 达到 `boundary_target` 的停止判断。
13. `agent_report.md` 不写“确认有效边界”。
14. stdout/stderr 摘要不泄露 API key。

16. 最小验收标准

第一版完成后必须满足：

1. 能通过 `AgentTask` 启动新实验。

2. 能通过 `AgentTask` 续跑已有实验。
3. 能只读复盘已有实验。
4. 能记录 Agent 状态、事件、观察和决策。
5. 能读取实验产物并生成摘要。
6. 能根据目标、pending、boundary 和预算停止。
7. 能输出 `agent_report.md`。
8. 能拒绝越权计划和越权决策。
9. 所有新增逻辑都有不访问真实 API 的测试。
10. 原有流水线入口仍能独立运行。

17. 人工 Review 清单

每阶段提交前检查：

- 是否把 Agent 改造和无关文档移动、归档工作混在一起。
- 是否新增了绕过 `pipeline_io.py` 的产物写入或覆盖。
- 是否让 Memory 直接决定算子或跳过 Router。
- 是否让控制层模型看到完整 JSONL、完整日志、参考答案全文或 Rubric 全文。
- 是否把 `report_only` 错写成 `execution_scope`。
- 是否在报告中把自动评分结果写成确认结论。
- 是否有真实 API 依赖进入单元测试。
- 是否修改现有流水线核心行为。
- 是否保留 `question_evolved=false` 的透传语义。
- 是否保持多算子搜索的“直接父子题比较”边界。

18. 推荐合并顺序

建议按以下顺序合并：

1. AgentTask、状态、事件、policy 骨架。
2. 确定性 planner 和 dry-run。
3. 工具封装与 subprocess 事件记录。
4. Observer 和 `agent_report.md`。
5. Decision Engine。
6. CLI 串联 `run`、`resume`、`review`。

7. 控制层模型接入。

8. 只读复盘 proposal。

这条顺序的核心原则是：先让 Agent 能安全记录、计划和观察，再让它调用现有流水线，最后才让控制层模型参与规划和决策。